



1



İçerik

- Sarmalılıyıcı
- Heap-Stack
- immutable – mutable
- Enum
- Metotlara default parametre yollama
- Lambda
- Jar Dosyası Oluşturma
- bitwise

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

2

Sarmalayıcı (Wrapper)

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

3

Sarmalayıcı (Wrapper)

- Sarmalayıcı sınıfları primitif (primitive) veri türleri olan **int**, **float**, **short**, **long** vs. gibi türleri birer nesne (object) olarak kullanmak için bize bir yol sunmaktadır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

4

Sarmalayıcı (Wrapper)

Primitif	Sarmalayıcı (Wrapper)
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

primitif tiplerin kullanılmadığı yerlerde, örneğin; ArrayList gibi koleksiyon nesneleriyle çalışırken sarmalayıcı sınıfları kullanmamız gerekir.

Bunun nedeni ArrayList yada diğer koleksiyonların yalnızca nesneleri saklayabilmesinden kaynaklanır. Primitif tipler birer nesne olarak geçmediği için koleksiyonla birlikte çalışmazlar.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

5

Heap-Stack

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

6

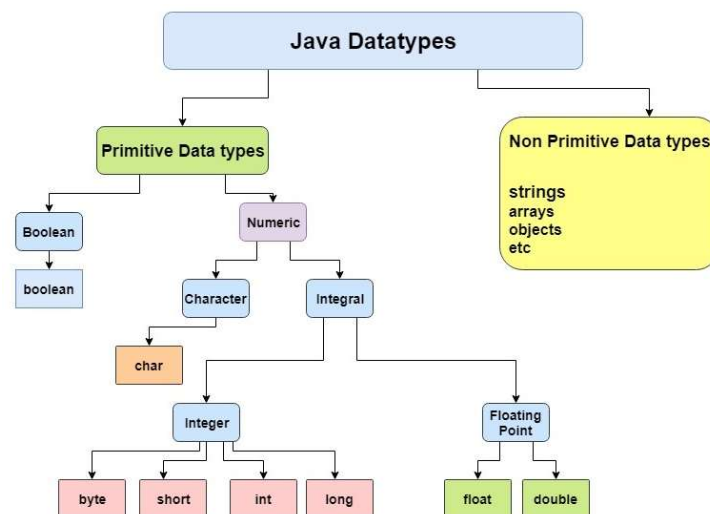
Heap-Stack

- Stack(Yığın) ve Heap bellekte (RAM) bulunan mantıksal yapılardır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

7

Heap-Stack



DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

8

Heap

- Java çalışma ortamı (Java Runtime) tarafından nesnelere ve JRE sınıflarına yer vermek için kullanılır.
- Yeni bir nesne oluşturulduğunda bu nesne her zaman heap memory içinde yer kaplar, oluşturulur.
- Sınıf değişkenleri heap içerisinde yer alır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

9

Stack

- Metodlar, metodlara gönderdiğimiz parametreler , local primitive(ilkel) tipler ve local referanslar(object reference) bulunur.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

10

Heap-Stack

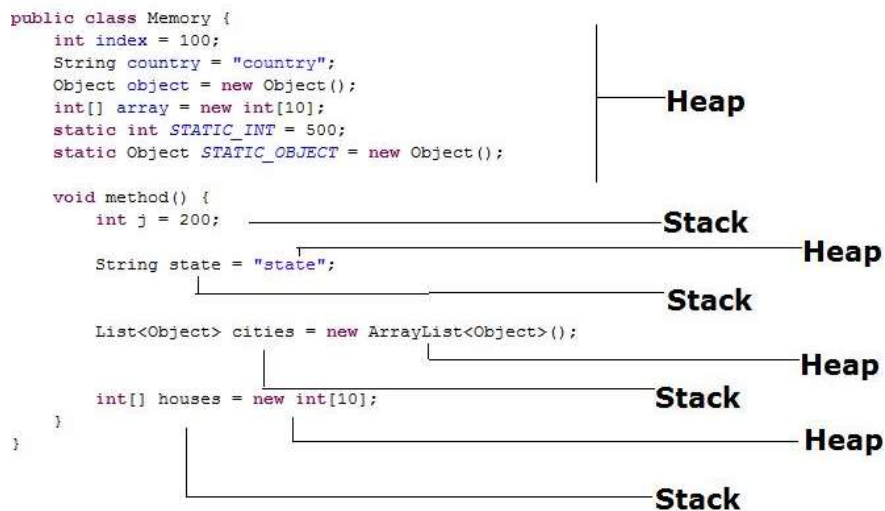
- Eğer primitive değişken class içerisindeyse **Heap** memory de, metod içerisindeyse **Stack** memory de tutulur

<http://serkansakinmaz.blogspot.com/2013/08/java-da-stack-ve-heap-kavram.html>

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

11

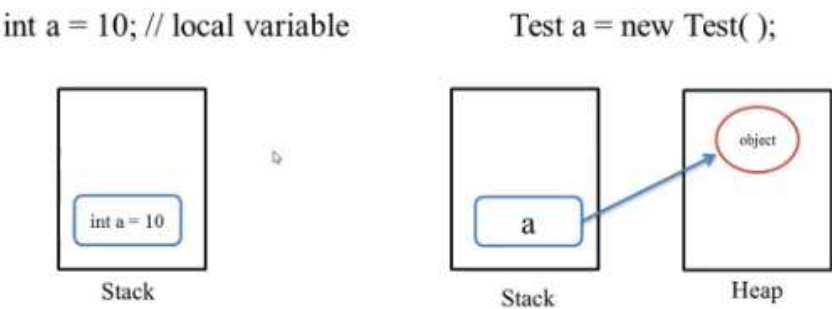
Heap-Stack



DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

12

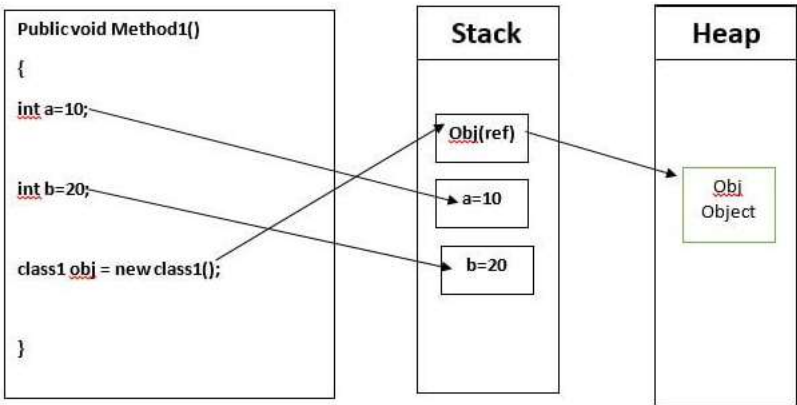
Heap-Stack



DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

13

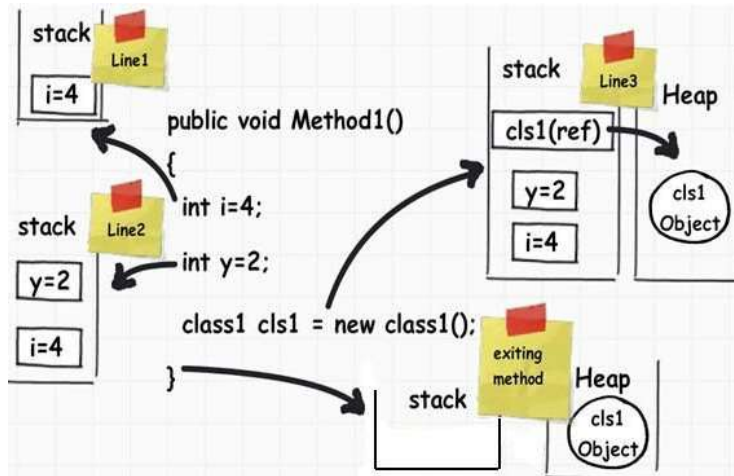
Heap-Stack



DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

14

Heap-Stack



DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

15

Heap-Stack

- Stack
 - Oluşturulan değişkenler *stack* kapsamından çıkınca otomatik olarak yok edilir.
 - Ulaşılması *heap*'e göre oldukça hızlıdır.
 - *Stack* üzerinde kullanım fazla olduğunda alan yeterli olmayabilir.
 - Oluşturulan değişkenler *pointer* olmadan kullanılabilir.
 - Derleme zamanında oluşturulur.
 - Kullanacağınız yerin boyutunu tam olarak biliyorsanız *Stack*'i kullanmak sizin için uygun olacaktır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

16

Heap-Stack

- Heap
 - Bir blok içerisinde oluşturulan *heap* değişkenler, bloğun dışına çıktığında otomatik olarak yok edilemez, bunun manuel olarak yapılması gerekir.
 - *Stack* ile karşılaştırıldığında oldukça yavaştır.
 - Doğru kullanılmaması durumunda bellek sorunları yaratır.
 - Değişkenler *pointer* ile kullanılır.
 - Çalışma zamanında oluşturulur.
 - İhtiyacınız olan boyutu tam olarak bilmiyorsanız *Heap* kullanımı sizin için biçilmiş kaftandır.

<https://medium.com/yigit-xcodeproj/stack-ve-heap-arasindaki-fark-nedir-stack-vs-heap-c61e3d463dd7>

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

17

Heap-Stack

- Stack memory'deki bellek yönetimi basitken ve LIFO mantığına dayanırken heap memory'deki bellek yönetimi çok daha karışıktır. Buna sebep olan şey Heap Memory'nin çok daha büyük olmasıdır. Heap memory'nin yönetimi için Garbage Collection algoritmaları kullanılır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

18

Heap-Stack

- Stack memory kısa ömürlü olurken heap memory uygulamanın başından sonunda kadar tutulur.
- Stack memory dolarsa **`java.Lang.StackOverflowError`** hatası, heap memory dolarsa **`java.Lang.OutOfMemoryError: Java Heap Space`** hatası alırız.
- Stack memory hem boyut olarak hem de bellek yönetimi olarak daha basit olduğundan heap memory'e göre daha hızlı hareket eder.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

19

Heap-Stack

- Heap de ki veri Garbage Collector (Çöp Toplayıcı) tarafından silinir.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

20

immutable – mutable değişmez – değişebilir

- Java'da hiçbir şey referansla iletilmez.
- Her şey değere göre iletilir.
- Nesne referansları değere göre iletilir.
- String ler değişmezdir(immutable-değişmez).
- String leri değişken yapmak istiyorsak StringBuffer kullanabiliriz.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

21

immutable – mutable değişmez – değişebilir

```
Date date = new Date();  
System.out.println(date);  
date.setDate(5);  
System.out.println(date);
```

```
Wed Jun 01 07:07:46 EET 2022  
Sun Jun 05 07:07:46 EET 2022
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

22

immutable - mutable değişmez - değişebilir

```
Date date = new Date();  
System.out.println(date);  
date.setDate(5);  
System.out.println(date);
```

```
Wed Jun 01 07:07:46 EET 2022  
Sun Jun 05 07:07:46 EET 2022
```

```
String str = "sdu";  
str.toUpperCase();  
System.out.println(str);
```

```
String st2=str.toUpperCase();
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

23

Enum

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

24

Enum

- enum (enumeration) yapısı, sabit değerleri daha anlamlı, güvenli ve yönetilebilir hale getirmek için kullanılan bir özelliktir.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

25

Enum

- Sabit Değerleri Gruplamak: Renkler, yönler, durumlar gibi sabit değerleri tek bir yapıda toplamak.
- Anlamlı Kod Yazmak: 1, 2 gibi anlamsız sabitler yerine DURUM.AKTIF gibi okunabilir kodlar kullanmak.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

26

Enum

- Hata Riskini Azaltmak: Yanlış veri girilmesini önler (örn. "aktif" yerine "akif" yazılması gibi yazım hataları engellenir).
- Switch-case ve karşılaştırmalarla kolay kullanım sağlar.
- Enum'a metod da tanımlanabilir, bu sayede daha zengin davranışlar sergileyebilir.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

27

Enum

```
enum Seviye {  
    DUSUK,  
    ORTA,  
    YUKSEK  
}  
  
Seviye seviye1 = Seviye.ORTA;  
  
enum Gun {  
    PAZARTESI, SALI, CARSAMBA, PERSEMBE,  
    CUMA, CUMARTESI, PAZAR  
}
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

28

Enum

- Örnek

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

29

Metotlara default parametre yollama

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

30

default parametre yollama

- Metot overload/aşırı yükleme ile yapılabilir.
- Metot parametre listesi Sınıf olarak tanımlanırsa-Sarmalıyıcı Sınıf- bu işlem gerçekleştirilebilir

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

31

Lambda

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

32

Lambda

- Kodun okunabilirliğini artırır.
- Daha az kod yazarak aynı işi yapmanızı sağlar.
- Özellikle Streams, Collections ve GUI olayları gibi yerlerde daha fonksiyonel bir yaklaşım sunar.
- Java'nın fonksiyonel arayüzleriyle (Runnable, Comparator, ActionListener, vb.)
- Koleksiyonlarla (Stream API)
- Event handling (örneğin JavaFX veya Swing)

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

33

Lambda

```
Runnable r = () -> System.out.println("Merhaba!");  
new Thread(r).start();
```

```
List<String> isimler = List.of("Ali", "Veli");  
isimler.forEach(isim -> System.out.println(isim));
```

```
button.setOnAction(e -> System.out.println("Butona  
tıklandı!"));
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

34

Switch-case 14+

```
int sayisalKarsilik = switch (gun) {  
    case PAZARTESI, CUMA, PAZAR -> 9;  
    case SALI -> 4;  
    default -> {  
        String s = gun.toString();  
        // Enum sabitinin ismini alır (örn: "CARSAMBA")  
        int sonuc = s.length();  
        // Harf sayısını hesaplar  
        yield sonuc;  
    }  
};
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

35

var 10+

- var lokal değişkenler için tip çıkarımı (type inference) sağlar.
- Sadece metot içindeki lokal değişkenlerde (yerel değişkenlerde) kullanılabilir.
- Sınıf üyeleri (field), method parametreleri veya dönüş tipleri için kullanılamaz.
- var sayi = 10;
 // sayi değişkeninin tipi int olur
- var metin = "Merhaba";
 // metin değişkeninin tipi String olur

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

36

Packages Import

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

37

Packages (Paketler)

- Java programları paketlerden (packages) oluşur.
- Paketlerin içinde sınıflar ve arayüzler (classes and interfaces) bulunur.
- Paketler dosya yapısını temsil eder.
- İyi bir programcı, birbirleriyle ilişkili olan sınıfları ve arayüzleri aynı pakete koyar.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

38

Packages (Paketler)

- Mantıksal olarak bağlantısı olan ya da başka bir deyişle birbiriyle ilişkili sınıflar aynı paket içinde yer almaktadırlar.
- Paket tanımı
`java.paketAdı.sınıfAdı`
şeklinde olur.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

39

Import

- **import** ise sınıfı tanımlar.
Yani eğer aynı pakette olmayan bir sınıf kullanılmak istenirse önce sınıfın import edilmesi gerekmektedir.
- Bazen sınıfAdı yerine * işareti ile karşılaşabilirsiniz.
Kullanımı şu şekildedir.
`import java.paketAdı.*;`
Bu, java.paketAdı paketindeki tüm sınıfları çağırabilirsın demektir.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

40

Packages / Import

Package Explorer

ilk_adim

src

package_1

sum_1.java

package_2

sum_2.java

Test_package

Main.java

JRE System Library [JavaSE-1]

Main.java

sum_1.java

sum_2.java

```
package package_1;

public class sum_1
{
    public static int sum(int a,int b)
    {
        return a+b;
    }
    public static void print()
    {
        System.out.println("package_1's print() function");
    }
}
```

Main.java

sum_1.java

sum_2.java

```
package package_2;

public class sum_2
{
    public static int sum(int a,int b)
    {
        return a+b;
    }
    public static void print()
    {
        System.out.println("package_2's print() function");
    }
}
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

41

Packages / Import

*Main.java

sum_1.java

sum_2.java

```
package Test_package;

import package_1.sum_1;
import package_2.sum_2;

public class Main
{
    public static void main(String[] args)
    {
        System.out.println("sum_1 = "+sum_1.sum(10,20));
        System.out.println("sum_2 = "+sum_2.sum(20,30));
        sum_1.print();
        sum_2.print();
    }
}
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

42

Turgay AYDOĞAN

21

Java Versions

- <https://www.marcobehler.com/guides/a-guide-to-java-versions-and-features>

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

43

JAR Dosyası Oluşturma

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

44

JAR Dosyası Oluşturma

- Çalıştırılabilir dosya.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

45

JAR Dosyası Oluşturma

- `jar -cvfm TextDeneme.jar MANIFEST.MF TextDeneme.class`

- Manifest.mf dosya içeriği
Manifest-version: 1.0
Main-Class: TextDeneme
Class-Path: TextDeneme.jar

Calistir.bat
`java -jar TextDeneme.jar`

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

46

bitwise

&	AND	
	OR	
^	XOR	Bitler farklıysa 1, aynıysa 0
~	NOT	Tüm bitleri tersine çevirir (1 → 0, 0 → 1)
<<	Sola kaydır	Bitleri sola kaydırır; sayı 2'nin katları şeklinde büyür
>>	Sağa kaydır	Bitleri sağa kaydırır; sayı 2'nin katları şeklinde küçülür
>>>	İşaretsiz sağa kaydır	Negatif sayılarda bile boşlukları 0 ile doldurarak sağa kaydırır

Bitwise operatörler, sayıları **ikili (binary)** düzeyde karşılaştırarak işlem yapar.

Yani sayıların bitlerini (0 ve 1) kullanarak doğrudan işlem gerçekleştirir.

Bu operatörler özellikle performans gerektiren uygulamalarda, sistem programlamada, donanım arayüzlerinde ve sıkıştırma/şifreleme işlemlerinde kullanılır.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

47

bitwise

```
int a = 5;           // 0101
int b = 3;           // 0011
int sonuc = a & b;   // 0001
```

```
System.out.println("a & b=" + sonuc);
// Sonuç: 1
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

48

bitwise

```
int a = 6;           // 0110
int b = 1;           // 0001
int sonuc = a | b;    // 0111

System.out.println("a | b=" + sonuc);
// Sonuç: 7
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

49

bitwise

```
int a = 9;           // 1001
int b = 5;           // 0101
int sonuc = a ^ b;    // 1100
System.out.println("a ^ b=" + sonuc);
// Sonuç: 12
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

50

bitwise

```
int a = 4; // 00000000 00000000 00000000 00000100
int b = ~a; // 11111111 11111111 11111111 11111011
```

```
System.out.println("~a = " + b); // Sonuç: -5
```

```
// tam sayılar 2'nin tümleyeni ile temsil edilir
// ~a = -(a + 1) olduğu için ~4 = -5
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

51

bitwise

```
int a = -8;
```

```
// Arithmetic right shift (işareti korur)
int sonuc1 = a >> 2;
```

```
// Logical right shift (işareti sıfırla doldurur)
int sonuc2 = a >>> 2;
```

```
System.out.println("a >> 2 = " + sonuc1);
// Sonuç: -2
```

```
System.out.println("a >>> 2 = " + sonuc2);
// Sonuç: 1073741822
```

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

52

bitwise

- `int a = -8`
- `// 32 bittir ve negatif sayılar 2'nin tümleyeni ile ifade edilir`
- `+8 = 00000000 00000000 00000000 00001000`
- `-8 = 11111111 11111111 11111111 11111000`
`// 2'nin tümleyeni`
- `>>` operatörü: Bitleri sağa kaydırır. En soldan boş kalan bitleri, mevcut işaret bitiyle (1) doldurur. Yani işaret korunur → sonuç yine negatif olur.

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

53

bitwise

- `a = 11111111 11111111 11111111 11111000 // -8`
- `a >> 2 = 11111111 11111111 11111111 11111110 // -2`
- (işaret biti 1 ile dolduruldu)

DR.ÖĞR.ÜYESİ TURGAY AYDOĞAN

54

bitwise

- >>> operatörü: Bitleri sağa kaydırır. En soldaki boş bitleri 0 ile doldurur → işaret yok sayılır. Yani sonuç pozitif olur çünkü artık işaret biti 0'dır.

- a = 11111111 11111111 11111111 1111000 // -8
- a >>> 2 = 00111111 11111111 11111111 1111110
- // pozitif sayı
- 00111111 11111111 11111111 1111110 = 1073741822

DR. ÖĞR. ÜYESİ TURGAY AYDOĞAN